Supporting Information

Class-Dependent Errors of Cheap Band-Gap Observables
Undermine High-Throughput Screening of Organic Photovoltaic Absorbers

Patrick Sorrel Mvoto Kongo, Steve Cabrel Teguia Kouam,
Jean-Pierre Tchapet Njafa, Serge Guy Nana Engo

Contents

S1 Validity rules of gate G0 **1**

S2 Salts, not halogens, at the window edges **1**

S3 Basis set and geometry of the Kohn–Sham gap **2**

S4 Simplified TDA on a B3LYP ground state **2**

S5 Sensitivity of the window count to calibration error **3**

S6 Molecules that pass the calibrated cascade **3**

S7 Formal verification **5**

S1 Validity rules of gate G0

Table S1 lists how many PubChemQC records each validity rule removes. Two rules dominate: 6 323 records fail the ring or aromaticity rules, and 838 records contain more than one fragment. The second group matters beyond its size, because neither gap observable describes a single absorber for a salt or a mixture (Section S2).

Table S1: Validity rules of gate G0, applied in the order listed; each record is counted under the first rule it fails. 10 215 of 17 458 records pass.

Rule	Record removed when the structure has	Records
R1	more than one fragment (salts mixtures co-crystals)	838
R2	contains a metal	2
R3	no ring	4 034
R4	no aromatic ring	2 289
R5	fewer than six heavy atoms	9
R6	fewer than six conjugated atoms	70
–	SMILES not parsable by RDKit	1

S2 Salts, not halogens, at the window edges

Halogenated records are more frequent near the absorber window than in the database as a whole, which could be read as a chemical trend. Table S2 shows that the excess follows the salts. The database contains no bromine or iodine; its chlorine is bound covalently, present as an isolated

fragment (chloride or HCl), or both. Covalently bound chlorine occurs in 2 of the 33 records of the Kohn–Sham window (6.1 %, both of them hydrochloride salts) and in the sTDA-xTB window at about its database frequency (13.4 against 14.7 %). The excess is carried by multi-fragment records: they make up 48.5 % of the Kohn–Sham window and 84.4 % of the records whose lowest sTDA-xTB singlet lies below 1 eV, consistent with spurious charge-transfer states between the separated ions. After Holm correction for the four enrichment tests performed, the halogen excess in the Kohn–Sham window is not significant ($p = 0.11$), while the excess among the collapsed records is ($p = 7\times10^{-127}$). Removing multi-fragment records (rule R1) therefore halves the Kohn–Sham window and removes most collapse cases.

Table S2: Composition of the 15 952 records with both observables and of three subsets. Percentages of each set. A record can count both as containing an isolated Cl fragment and as containing covalent Cl. Odds ratio and Holm-adjusted Fisher p for halogenated records in the subset against the rest of the database.

Set	n	Halogenated	Multi-fragment	Isolated Cl fragment	Covalent Cl	Odds ratio	p_{Holm}
All records	15 952	21.7	4.5	2.9	14.7	–	–
E_{KS} in 1.1 eV to 2.2 eV	33	36.4	48.5	36.4	6.1	2.06	0.11
$E_{\mathrm{S_1}}$ in 1.1 eV to 2.2 eV	559	22.2	10.0	5.9	13.4	1.03	0.79
$E_{\mathrm{S_1}} < 1$ eV	742	61.5	84.4	53.9	12.0	6.46	7×10^{-127}

S3 Basis set and geometry of the Kohn–Sham gap

The calibration set uses B3LYP/def2-SVP gaps on DFT geometries, the database stores B3LYP/6-31G* gaps on PM6 geometries, and our recomputations used force-field (UFF) geometries. Table S3 compares these levels for the acenes. For pentacene at a fixed UFF geometry, def2-SVP and 6-31G* give gaps of 1.884 and 1.896 eV: the basis set moves the gap by 0.01 eV, far less than the offsets discussed in the main text. Geometry moves it more: re-optimising pentacene at B3LYP/6-31G* raises its gap to 2.209 eV, 0.31 eV above the UFF value at the same level. Across the four acenes, the UFF/def2-SVP gaps lie 0.37–0.51 eV below the database values, a difference that combines geometry and basis. On DFT or PM6 geometries the Kohn–Sham offset to the measured L_a energy ranges from about zero (pentacene, own optimisation) to +0.46 eV (naphthalene, database value), against +0.29 eV for the donor–acceptor molecules of HOPV15, a smaller class contrast than for sTDA-xTB, which reproduces the acenes to 0.12 eV on UFF geometries but overestimates the donor–acceptor gaps by 1.0 eV. No sTDA-xTB calculation on DFT geometries of the acenes was made, so the size of the sTDA-xTB contrast after a geometry correction is not quantified here.

Table S3: Kohn–Sham gaps of four acenes [eV]. Experimental values: midpoints of the measured lowest bright (L_a) excitation energies (Malloci et al., 2007). Offset: Kohn–Sham gap minus experiment.

Acene	def2-SVP, UFF	6-31G*, PM6 (PubChemQC)	Exp. L_a	Offset, def2-SVP	Offset, PubChemQC
Naphthalene	4.537	4.909	4.445	+0.092	+0.464
Anthracene	3.247	3.709	3.360	−0.113	+0.349
Tetracene	2.431	2.931	2.660	−0.229	+0.271
Pentacene	1.884	2.397	2.215	−0.331	+0.182

S4 Simplified TDA on a B3LYP ground state

Interface. std2 reads the ground state from a Molden file. Two details decide whether the result is correct: the `[Atoms]` block must be written in Ångström (`[Atoms] Angs`), because the program misreads coordinates labelled in atomic units and then reports a singular overlap matrix; and the molecular-orbital coefficients must refer to Cartesian Gaussian functions, so the PySCF calculation

was run with `cart=True`. With these settings std2 gave a pentacene singlet of 2.094 eV (sTDA) against 2.203 eV from a full TDA-B3LYP/6-31G* calculation in PySCF on the same geometry, a difference of 0.11 eV. Excitation energies were re-extracted from every std2 output with a parser that requires the state-table header; all 36 molecules parsed.

Result. Table S4 compares the observables on the 36 HOPV15 molecules with DFT geometries. Moving sTDA onto a B3LYP ground state removes the 0.76 eV overestimate of sTDA-xTB, which points to the tight-binding ground state as the main source of that bias. In this set, sTDA@B3LYP is then the most accurate gap observable *without* calibration, with 28 of 31 molecules recalled by the raw window. Its molecule-to-molecule scatter is larger than that of E_{KS}, however, and after a linear calibration the stored Kohn–Sham gap is better by 0.06 eV (paired bootstrap 95 % interval 0.031–0.091 eV, 34 complete cases). The exact-integral variant XsTDA brings no gain here. For a screen of this class, a calibrated E_{KS}, already stored in the database, is the more accurate choice; sTDA@B3LYP is useful as an uncalibrated cross-check.

Table S4: Gap observables computed on the DFT geometries distributed with HOPV15, against the experimental optical gap ($n = 36$ donor–acceptor small molecules). SD: standard deviation of the signed error. Recall: molecules whose experimental and computed gaps both lie in the 1.1 eV to 2.2 eV window. LOO MAE: leave-one-out error of a linear calibration against experiment.

Observable	Bias [eV]	MAE [eV]	SD [eV]	r	Recall	LOO MAE [eV]
E_{KS}, B3LYP/def2-SVP	+0.29	0.29	0.16	0.92	23/31	0.083
sTDA@B3LYP/6-31G*	-0.02	0.21	0.26	0.75	28/31	0.141
XsTDA@B3LYP/6-31G*	+0.24	0.25	0.29	0.69	23/31	0.157
sTDA-xTB	+0.76	0.76	0.12	0.89	0/31	0.103

S5 Sensitivity of the window count to calibration error

The calibrated window edges carry the uncertainty of the calibration. Shifting the window by plus or minus the leave-one-out root-mean-square error of the calibrated E_{KS} (0.115 eV) changes the number of valid records inside the calibrated Kohn–Sham window from 46 to between 34 and 58. In this window-shift test the count moves by about ±25 %, so counts produced by the gap step should not be read to the unit. The upper edge is a convention as well: without the window, 566 molecules of the cascade have a model efficiency of at least 1.86 %, the lowest value among the 37 survivors, and 31 of them reach 5 % or more (calibrated optical gaps 2.23 eV to 2.58 eV, maximum 8.34 %).

S6 Molecules that pass the calibrated cascade

Table S5 lists the 37 survivors. Two entries and one group illustrate what the cascade does and does not catch. Pentacene (ID 8671) passes, with a calibrated gap of 2.02 eV. The anthraquinone dye 22936 passes; no gate tests reactivity or photostability, so dyes of this kind are not excluded. Seven survivors have $E_{\mathrm{S_1}}$ more than 0.5 eV below E_{opt}; the record check flags only the opposite disagreement and leaves them in. Only 10 of the 37 lie within the chemical domain of the calibration set, as defined by the six-descriptor Mahalanobis criterion of the Methods; the calibrated gaps of the other 27 carry an error that the in-class leave-one-out error does not describe. The table, with the full SMILES, is also deposited as `si_survivors.csv`.

Table S5: The 37 molecules that pass every gate of the calibrated cascade, ordered by Scharber efficiency (upper bound, PCBM acceptor). E_{KS}: stored PubChemQC gap; E_{opt}: calibrated optical gap; E_{S_1}: sTDA-xTB singlet energy; $\mathrm{PCE}_{0.65}$ and $\mathrm{PCE}_{\mathrm{G}}$: efficiency with $FF = 0.65$ and with Green's fill factor; d_{M}: Mahalanobis distance to the calibration set, * marking the 27 molecules beyond the domain threshold ($d_{\mathrm{M}} > 4.29$). Energies in eV, V_{OC} in V, efficiencies in %.

ID	Formula	SMILES	E_{KS}	E_{opt}	E_{S_1}	V_{OC}	$\mathrm{PCE}_{0.65}$	$\mathrm{PCE}_{\mathrm{G}}$	d_{M}
17574	C23H19N3O6S	CC(=O)N(c1ccc(cc1)Nc1cc(c(c2c1C(=O)c1ccccc1C2=O)N)S(=O)(=O)O)C	2.33	1.98	1.66	1.60	10.1	14.3	4.3*
12919	C13H10N4O4	O=Nc1ccc(=O)c(cc1)N=NC(=c1ccn(cc1)O)O	2.08	1.81	2.11	1.20	9.9	13.6	7.2*
12913	C13H10N4O3	O=Nc1ccc(c(=O)cc1)NNC(=O)c1ccncc1	2.33	1.98	1.13	1.51	9.6	13.6	7.0*
18506	C20H17N3O4	CCCCn1c(=O)c2c(c1=O)c(N)c1c(c2N)c(=O)c2c(c1=O)cccc2	2.26	1.93	1.50	1.39	9.5	13.3	3.0
22936	C22H17N3O6S	CC(=O)Nc1ccc(cc1)Nc1cc(c(c2c1C(=O)c1ccccc1C2=O)N)S(=O)(=O)O	2.04	1.78	1.73	1.10	9.4	12.8	4.7*
327	C14H8N2O8	OC(=O)c1cc(C(=O)O)c2c([nH]1)c(O)c(=O)c1c2[nH]c(c1)C(=O)O	2.42	2.04	1.65	1.59	9.2	13.0	8.7*
22916	C20H14N2O5S	O=C1c2ccccc2C(=O)c2c1c(Nc1ccccc1)cc(c2N)S(=O)(=O)O	2.29	1.95	1.70	1.39	9.2	12.8	4.2
19385	C8H7N3O2S	Nc1ccc(c(c1)O)C1N=NC(=S)O1	2.51	2.10	1.52	1.68	8.8	12.5	7.0*
22902	C23H20N2O5S	Cc1cc(C)c(c(c1)C)Nc1cc(c(c2c1C(=O)c1ccccc1C2=O)N)S(=O)(=O)O	2.47	2.07	1.78	1.46	8.1	11.3	2.8
6689	C20H15N3O8S2	Nc1c2C(=O)c3ccccc3C(=O)c2c(cc1S(=O)(=O)O)Nc1ccc(c(c1)S(=O)(=O)O)N	2.37	2.01	1.69	1.31	8.0	11.1	6.0*
5773	C19H17N3O4S2	O=C(N[C@@H]1C(=O)N2[C@@H]1SCC(=C2C(=O)[O-])C[n+]1cccc1)Cc1cccs1	2.35	1.99	1.85	1.26	7.8	10.8	4.2
6690	C28H14N2O4	O=c1c2c3[nH]c4ccc5c(c4[nH]c3ccc2c(=O)c2c1cccc2)c(=O)c1c(c5=O)cccc1	2.26	1.93	2.36	1.09	7.5	10.3	3.2
5570	C7H7NO2	C[n+]1cccc(c1)C(=O)[O-]	2.56	2.13	1.94	1.20	6.0	8.3	4.9*
20549	C18H18N2O4	OCCNc1ccc(c2c1C(=O)c1c(C2=O)cccc1)NCCO	2.53	2.11	1.55	1.15	6.0	8.2	3.8
17157	C7H7NO2	[O-]C(=O)c1cc[n+](cc1)C	2.55	2.13	1.93	1.18	6.0	8.2	4.9*
16356	C9H8N2O	CNC1=c2ccccc2=NC1=O	2.56	2.13	0.37	1.19	5.9	8.2	4.8*
18514	C18H18N2O6	OCCNc1ccc(c2c1C(=O)c1c(C2=O)c(O)ccc1O)NCCO	2.44	2.05	1.86	1.03	5.8	7.9	4.7*
17191	C17H16N2O3	OCCNc1ccc(c2c1C(=O)c1ccccc1C2=O)NC	2.54	2.12	1.54	1.11	5.6	7.7	3.4
6817	C13H10N2O	Cn1c2ccccc2nc2c1cccc2=O	2.63	2.18	2.34	1.19	5.5	7.6	5.1*
6680	C21H15NO3	Cc1ccc(cc1)Nc1ccc(c2c1C(=O)c1ccccc1C2=O)O	2.62	2.18	1.82	1.17	5.5	7.6	4.1
11296	C11H8O5	Oc1c(=O)cc2c(c(c1O)O)c(O)ccc2	2.34	1.98	1.85	0.80	5.1	6.7	5.6*
9406	C32H18	c1ccc2c(c1)cc1c(c2)cc2c(c1)cc1c(c2)c2cccc3c2c2c1ccccc2cc3	1.99	1.75	2.42	0.56	5.0	6.2	6.0*
2134	C22H28N4O4	OCCNCCNc1ccc(c2c1C(=O)c1c(C2=O)cccc1)NCCNCCO	2.57	2.14	1.54	0.97	4.8	6.5	6.4*
10719	C14H16	CCc1ccc(c2c(c1)c(C)cc2)C	2.20	1.89	1.45	0.66	4.8	6.2	8.8*
20419	C20H14N2O2	O=C1c2ccccc2C(=O)c2c1c(ccc2N)Nc1ccccc1	2.56	2.13	1.74	0.93	4.7	6.3	4.6*
4212	C22H28N4O6	OCCNCCNc1ccc(c2c1C(=O)c1c(C2=O)c(O)ccc1O)NCCNCCO	2.53	2.11	1.79	0.86	4.5	5.9	6.9*

ID	Formula	SMILES	E_{KS}	E_{opt}	E_{S_1}	V_{OC}	$PCE_{0.65}$	PCE_G	d_M
9139	C32H18	c1ccc2c(c1)cc1c(c2)cc2c(c1)cc1c(c2)cc2c3c1ccc1c3c(cc2)ccc1	2.18	1.88	2.21	0.60	4.4	5.7	6.0*
14650	C15H12N2O2	CNc1ccc(c2c1C(=O)c1ccccc1C2=O)N	2.63	2.18	1.73	0.90	4.2	5.6	4.4*
9105	C32H18	c1ccc2c(c1)cc1c(c2)cc2c3c1ccc1c3c(cc2)c2c(c1)cc1c(c2)cccc1	2.36	2.00	2.53	0.67	4.1	5.3	6.0*
8671	C22H14	c1ccc2c(c1)cc1c(c2)cc2c(c1)cc1c(c2)cccc1	2.40	2.02	2.23	0.68	4.1	5.3	7.2*
10881	C7H6O	O=c1cccccc1	2.32	1.97	2.77	0.62	4.0	5.1	7.2*
17189	C16H14N2O2	CNc1ccc(c2c1C(=O)c1ccccc1C2=O)NC	2.51	2.10	1.60	0.75	3.9	5.2	3.9
19729	C16H14N2O4	CNc1ccc(c2c1C(=O)c1c(C2=O)c(NC)ccc1O)O	2.53	2.11	2.05	0.75	3.8	5.0	4.3
22937	C26H18N2O6	Oc1ccc(cc1)Nc1ccc(c2c1C(=O)c1c(C2=O)c(O)c(cc1)O)Nc1ccc(cc1)O	2.45	2.06	1.72	0.50	2.8	3.5	4.6*
17190	C14H12N4O2	O=C1c2c(N)ccc(c2C(=O)c2c1c(N)ccc2N)N	2.56	2.13	1.95	0.55	2.7	3.4	5.5*
19428	C20H14	C1CC=c2c(=C1)cc1c3c2cc2cccc2c3C=C1	2.46	2.07	2.24	0.48	2.7	3.3	7.6*
9175	C22H16	c1ccc2=CC3C(=c4cc5ccccc5cc4=CC3)C=c2c1	1.74	1.57	1.44	0.17	1.9	1.7	7.5*

S7 Formal verification

The three statements of the Methods were proved in Lean 4 (v4.34.1) with Mathlib. The build completes with no `sorry`, no `admit` and no added axiom. An independent reader compared each formal statement with its intended meaning and found all three faithful.

F1, order invariance. `successiveFilter s ps` applies the pass/fail predicates `ps` to the finite set `s` one after another. `successiveFilter_eq_filter` shows that the result is the set of elements passing every predicate; `successiveFilter_perm` shows that any reordering of the predicates gives the same set. Order therefore changes only how many records each gate is credited with removing. The theorem does not say whether a gate's observable is meaningful for a record; that is the role of the validity gate (Section S2).

```
import Mathlib.Data.Finset.Filter
import Mathlib.Data.List.Perm.Basic
import Mathlib.Tactic.Tauto

open scoped Classical

namespace Formal.F01

variable {α : Type*}

/-- Apply each predicate in the list, in order. Classical decidability also
allows lists of arbitrary predicates, hence in particular decidable ones. -/
noncomputable def successiveFilter (s : Finset α) : List (α → Prop) → Finset α
  | [] => s
  | p :: ps => successiveFilter (s.filter p) ps

theorem successiveFilter_eq_filter (s : Finset α) (ps : List (α → Prop)) :
    successiveFilter s ps = s.filter (fun x => ∀ p ∈ ps, p x) := by
  induction ps generalizing s with
```

```
  | nil => ext x; simp [successiveFilter]
  | cons p ps ih =>
    rw [successiveFilter, ih]
    ext x
    simp only [Finset.mem_filter, List.mem_cons, forall_eq_or_imp]
    tauto

theorem successiveFilter_perm (s : Finset α) {ps qs : List (α → Prop)}
    (h : ps.Perm qs) : successiveFilter s ps = successiveFilter s qs := by
  rw [successiveFilter_eq_filter, successiveFilter_eq_filter]
  ext x
  simp only [Finset.mem_filter]
  constructor
  · rintro ⟨hx, hp⟩
    exact ⟨hx, fun p hq => hp p (h.mem_iff.mpr hq)⟩
  · rintro ⟨hx, hq⟩
    exact ⟨hx, fun p hp => hq p (h.mem_iff.mp hp)⟩

end Formal.F01
```

F2, robustness across conventions. `Robust C Adm` is the set of molecules admitted under every convention `c` of a finite family `C`. `mem_Robust` and `Robust_subset` state that it is exactly the intersection of the admitted sets, and `Robust_empty_example` gives a two-convention family for which it is empty: a molecule that survives every convention need not exist.

```
import Mathlib.Data.Finset.Basic
import Mathlib.Data.Fintype.Basic
import Mathlib.Tactic.NormNum

namespace Formal.F02

variable {ι α : Type*}

/-- The admissible elements common to every convention in the finite index set. -/
def Robust (C : Finset ι) (Adm : ι → Finset α) : Set α :=
  {x | ∀ c ∈ C, x ∈ Adm c}

theorem mem_Robust (C : Finset ι) (Adm : ι → Finset α) (x : α) :
    x ∈ Robust C Adm ↔ ∀ c ∈ C, x ∈ Adm c := Iff.rfl

theorem Robust_subset (C : Finset ι) (Adm : ι → Finset α) {c : ι} (hc : c ∈ C) :
    Robust C Adm ⊆ (Adm c : Set α) := by
  intro x hx
  exact hx c hc

/-- Even two nonempty admissible sets can have empty robust intersection. -/
theorem Robust_empty_example :
    Robust (Finset.univ : Finset Bool)
      (fun c => if c then ({0} : Finset ℕ) else {1}) = ∅ := by
  ext x
  simp only [Robust, Set.mem_setOf_eq, Set.mem_empty_iff_false, iff_false]
  intro h
  have h0 := h true (Finset.mem_univ true)
  have h1 := h false (Finset.mem_univ false)
  norm_num at h0 h1
  exact Nat.zero_ne_one (h0.symm.trans h1)

end Formal.F02
```

F3, Green's fill factor. With v the normalised open-circuit voltage, $FF(v) = \big(v - \ln(v + 0.72)\big)/(v + 1)$. `FF_bounds` proves $0 < FF(v) < 1$ for $v \geq 0$, and `FF_strictMonoOn` proves that FF increases strictly for $v \geq 1$. Negative v is not covered.

```
import Mathlib.Analysis.SpecialFunctions.Log.Deriv
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Positivity

namespace Formal.F03

noncomputable def FF (v : ℝ) : ℝ :=
  (v - Real.log (v + 0.72)) / (v + 1)

theorem FF_bounds {v : ℝ} (hv : 0 ≤ v) : 0 < FF v ∧ FF v < 1 := by
  have hy : 0 < v + (0.72 : ℝ) := by linarith
  have hd : 0 < v + 1 := by linarith
  have hu := Real.log_le_sub_one_of_pos hy
  have hl := Real.one_sub_inv_le_log_of_pos hy
  have hi : (v + (0.72 : ℝ))⁻¹ < 2 := by
    rw [inv_eq_one_div, div_lt_iff₀ hy]
    linarith
  constructor
  · exact div_pos (by linarith) hd
  · unfold FF
    apply (div_lt_one hd).2
    linarith

/-- The quotient rule, with its derivative kept in a form suitable for bounds. -/
theorem hasDerivAt_FF {v : ℝ} (hv : 0 ≤ v) :
    HasDerivAt FF
      (((1 - 1 / (v + 0.72)) * (v + 1) - (v - Real.log (v + 0.72))) /
        (v + 1) ^ 2) v := by
  have hy : v + (0.72 : ℝ) ≠ 0 := by linarith
  have hd : v + (1 : ℝ) ≠ 0 := by linarith
  unfold FF
  simpa only [id_eq, mul_one] using
    ((hasDerivAt_id v).fun_sub (((hasDerivAt_id v).add_const (0.72 : ℝ)).log hy)).fun_div
      ((hasDerivAt_id v).add_const (1 : ℝ)) hd

theorem FF_strictMonoOn : StrictMonoOn FF (Set.Ici 1) := by
  apply strictMonoOn_of_deriv_pos (convex_Ici 1)
  · intro v hv
    exact (hasDerivAt_FF (by have := hv; simp only [Set.mem_Ici] at this;
    ↪  linarith)).continuousAt.continuousWithinAt
  · intro v hv
    have hv1 : 1 < v := by simpa only [interior_Ici, Set.mem_Ioi] using hv
    have hv0 : 0 ≤ v := by linarith
    have hy : 0 < v + (0.72 : ℝ) := by linarith
    have hl := Real.one_sub_inv_le_log_of_pos hy
    have hi : 1 / (v + (0.72 : ℝ)) < (3 / 4 : ℝ) := by
      apply (div_lt_iff₀ hy).2
      linarith
    have hmul : (1 / (v + (0.72 : ℝ))) * (v + 0.72) = 1 :=
      div_mul_cancel₀ 1 (ne_of_gt hy)
    rw [(hasDerivAt_FF hv0).deriv]
    apply div_pos _ (sq_pos_of_pos (by linarith))
    rw [inv_eq_one_div] at hl
    nlinarith
```

```
end Formal.F03
```